

Code-Layout, Readability and Reusability

Date	3 July 2026
Team ID	
Project Name	Voice-Based Communication Understanding Analyzer
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Python follows 4-space indentation consistently.
2	Proper File Structure	Files and folders are logically organized	Yes	Separate Python modules for transcription, semantic analysis, report generation, and scoring.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as <code>audio_path</code> , <code>transcription</code> , <code>similarity_score</code> , and <code>waveform_image</code> improve readability.
4	Function / Method Names	Functions are descriptively named	Yes	Functions like <code>transcribe_audio()</code> , <code>generate_pdf()</code> , and <code>evaluate_semantics()</code> clearly describe their purpose.
5	Code Comments	Inline and block comments explain logic	Partial	Comments are provided for important functions but can be expanded further.
6	Modular Design	Code is split into reusable functions/modules	Yes	Independent modules for speech recognition, semantic evaluation,

				scoring, waveform generation, and reporting.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared functionality is reused across modules, minimizing code duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Uses try-except blocks for audio processing, file handling, and report generation.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technolog	yWhere Reused	Reusability Level (High / Medium / Low)
1	speech_to_text.py	Python, OpenAI Whisper	Speech transcription	High
2	semantic_eval.py	Python, Sentence-BERT	Concept similarity analysis	High
3	audio_features.py	Python, Librosa	Speech feature extraction	High
4	report_generator.py	Python, ReportLab	PDF report generation	High
5	waveform_generator.py	Python, Matplotlib	Waveform visualization	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Well-organized modular architecture with separate functional modules.
Readability	5	Clear naming conventions, simple logic flow, and consistent formatting.
Reusability	5	Modules are reusable and independent, enabling future enhancements.
Documentation / Comments	4	Core functions are documented; additional inline comments could further improve maintainability.
Overall Score	4.8/5	The project follows good software engineering practices with modular, readable, and maintainable code suitable for future extension.